Lycée Secondaire Hammam-Sousse	**Devoir de Synthèse n°2**	**Matière : Informatique** **Classe :** 2TI - **Durée :** 2h **Enseignants :** M MANI, Mme Achour, Mme Chtioui **Année Scolaire :** 2024-2025

Nom & Prénom : .....

Note / 20

Exercice 1 (4 Points)

Soit la fonction **f1** suivante :

```
Fonction f1(n: ……):……
Début
  ch ← ""
  Si n = 0 Alors
    ch ← "0"
  Fin Si
  Tant Que (n != 0) Faire
    ch ← ConvCh(n mod 2) + ch
    n ← n div 2
  Fin Tant Que
  Retourner ch
Fin
```

T.D.O.L

Objet	Type

1) Compléter l'entête de la fonction **f1**, ainsi que le **T.D.O.L**.
2) Retrouver le résultat de l'appel de la fonction **f1** pour les valeurs suivantes de **n** :

n	f1(n)
6	
19	

3) En déduire le rôle de la fonction **f1**.

   .....

4) **Exploiter la fonction f1** pour compléter la procédure **Conversion** qui convertit les **n** éléments du tableau **t1** ($5 \leq n \leq 100$) en binaire, le résultat de la conversion est stocké dans le tableau **t2**.

```
Procédure Conversion(……)
Début

  ……

  ……

  ……

  ……

  ……
Fin
```

T.D.O.L

Objet	Type

T.D.N.T
tab1 = tableau de 100 entier tab2 = tableau de 100 chaîne

Exercice 2 (3 Points)

N personnes se défient entre-elles durant les éliminatoires d'un tournoi de jeu d'échecs.

Lors de l'inscription, chaque joueur obtient son numéro.

L'algorithme ci-dessous permet de mélanger les numéros des joueurs, puis d'afficher ceux qui vont s'affronter dans chaque partie.

```
Algorithme Tournoi_Échecs
Début
  // Étape 1 - Saisir le nombre de joueurs
  Écrire("Nombre de joueurs ? ") ; Lire(n)
  // Étape 2 - Mélanger les numéros des joueurs
  Mélanger(tj, n)
  // Étape 3 - Afficher les parties
  Afficher(tj, n)
Fin
```

TDNT
tab = tableau de 100 entier

T.D.O.G

Objet	Type
n	Entier
tj	Tab
Mélanger Afficher	Procédure

```
Procédure Mélanger(@t: tab, nj: entier)
Début
  Pour i de 0 à nj-1 Faire
    t[i] ← i+1
  Fin Pour
  Pour i de 0 à nj-1 Faire
    p ← aléa(0, n-1)
    Permuter(t[i], t[p])
  Fin Pour
Fin
```

T.D.O.L

Objet	Type
i, p	Entier
Permuter	Procédure

1) Identifier le type de paramètres dans les algorithmes ci-dessus :
   - Souligner tous les **paramètres formels**.
   - Encadrer tous les **paramètres effectifs**.
2) Retrouver les **paramètres** de la procédure **Mélanger** ainsi que le **mode de passage** de chacun d'eux.

   .....

   .....

   .....

3) Transformer l'**étape 1** en un module. **Effectuer le contrôle de saisie** :
   - n doit être un nombre pair
   - $n \in [6, 100]$

   .....

   .....

   .....

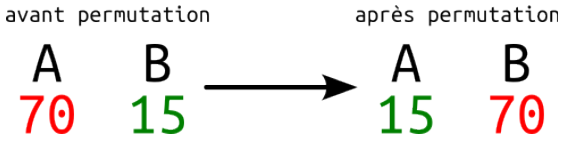
   .....

   .....

   .....

   .....

4) Développer l'algorithme de la procédure **Permuter** qui permet d'échanger le contenu des variables passées en paramètre.



.....

.....

.....

.....

.....

.....

.....

Exercice 3 (3 Points)

Soit la séquence suivante, **n** étant un entier strictement positif :

```
s ← 0
i ← n - (n + 1) mod 2
Tant Que i > 0 Faire
  s ← s + i
  i ← i - 2
Fin Tant Que
Écrire(s)
```

n = 5

s	**i**	**i > 0**

n = 8

s	i	i > 0

1) **Retrouver la valeur affichée** par cette séquence pour les valeurs ci-dessous. **Donner la trace d'exécution.**

n	**s**
5	
8	

2) La séquence ci-dessus présente un choix inapproprié de la structure itérative utilisée (Tant Que … Faire). **Récrire cette séquence** en utilisant une structure itérative plus adéquate.

.....

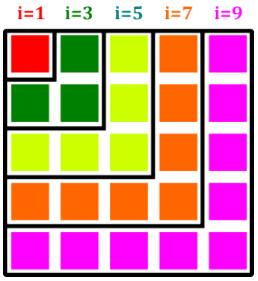
.....

.....

.....

.....

3) Sachant que la séquence ci-dessus calcule la **somme des nombres impairs inférieurs ou égaux à n**. Trouver une expression simplifiée qui permet de retrouver **s** en fonction de **n**, sans utiliser de boucle.




```
fonction somme_impairs(n: entier):entier
Début
  s ← ..................................................
  Retourner s
Fin
```

Problème (10 Points)